Processor Design Using Logisim

Multiplication Operation using 8 bit register files

Name : Chinmay N. Yalawatti

USN : 01FE24BEC258

Course : Computer Organization and Architecture

Division : E- Div

1. Objective

The objective of this project is to design and simulate a basic processor using Logisim that can perform fundamental operations such as data transfer, arithmetic, logic, and memory access. The processor is built using digital logic components to understand:

- How instructions are fetched, decoded, and executed
- The role of registers, ALU, memory, and control signals
- The internal data flow inside a processor

This project helps in visualizing Computer Organization and Architecture (COA) concepts practically.

---

2. Processor Architecture:

The designed processor follows a simple single-cycle architecture implemented using Logisim components.

Architecture Type

- Register-based processor
- Shared data bus architecture
- Single ALU
- Centralized control logic

Major Architectural Blocks

- Program Counter (PC)
- Instruction Memory
- Instruction Register (IR)
- Register File
- Arithmetic Logic Unit (ALU)
- Data Memory
- Control Unit
- Multiplexers and buses

All components are connected using common data and control buses.

---

3. Instructions

The processor supports a basic instruction set. Each instruction is executed in one complete cycle.

Instruction Categories

1. **Data Transfer Instructions**
    - LOAD (Memory → Register)
    - STORE (Register → Memory)
    - MOV (Register → Register)
2. **Arithmetic Instructions**
    - MUL
3. **Logical Instructions**
    - AND
    - OR
    - XOR
4. **Control Instructions**
    - NOP / simple flow control (implicit)

Instruction Format (General)

```
Opcode → Source Register →Destination Register →Immediate→ Address
```

The opcode controls ALU operation, memory access, and register write enable.

---

4. Key Components

4.1 Program Counter (PC)

- Holds the address of the current instruction
- Automatically increments after each instruction
- Connected to instruction memory address input

4.2 Instruction Memory

- Stores the program instructions
- Addressed by the PC
- Outputs instruction to the Instruction Register

4.3 Instruction Register (IR)

- Temporarily stores the fetched instruction
- Separates opcode and operand fields
- Feeds the control unit and register selectors

4.4 Register File

- Contains multiple general-purpose registers
- Supports:
    - One read ports
    - One write port
- Register selection is done using multiplexers

4.5 Arithmetic Logic Unit (ALU)

- Performs arithmetic and logical operations
- Operations selected using ALU control signals
- Produces:
    - Result
    - Status outputs (optional)

4.6 Data Memory

- Used for LOAD and STORE operations
- Controlled by:
    - `Mem_Read`
    - `Mem_Write`
- Data comes from ALU or register file

4.7 Control Unit

- Decodes opcode from IR
- Generates control signals such as:
    - Register write enable
    - ALU operation select
    - Memory read/write
    - MUX select signals

This is implemented using **logic gates and control wiring**.

4.8 Multiplexers

- Used extensively for:
    - Register selection
    - ALU input selection
    - Write-back data selection

---

5. Working

The processor works in the following **step-by-step manner**:

Step 1: Instruction Fetch

- PC sends address to instruction memory
- Instruction is fetched and stored in IR

Step 2: Instruction Decode

- Opcode is decoded by the control unit
- Control signals are generated
- Source and destination registers are selected

Step 3: Operand Fetch

- Register file outputs operands
- Data is sent to ALU inputs via multiplexers

Step 4: Execution

- ALU performs the required operation
- For memory instructions, ALU calculates address

Step 5: Memory Access

- LOAD → data read from memory
- STORE → data written to memory

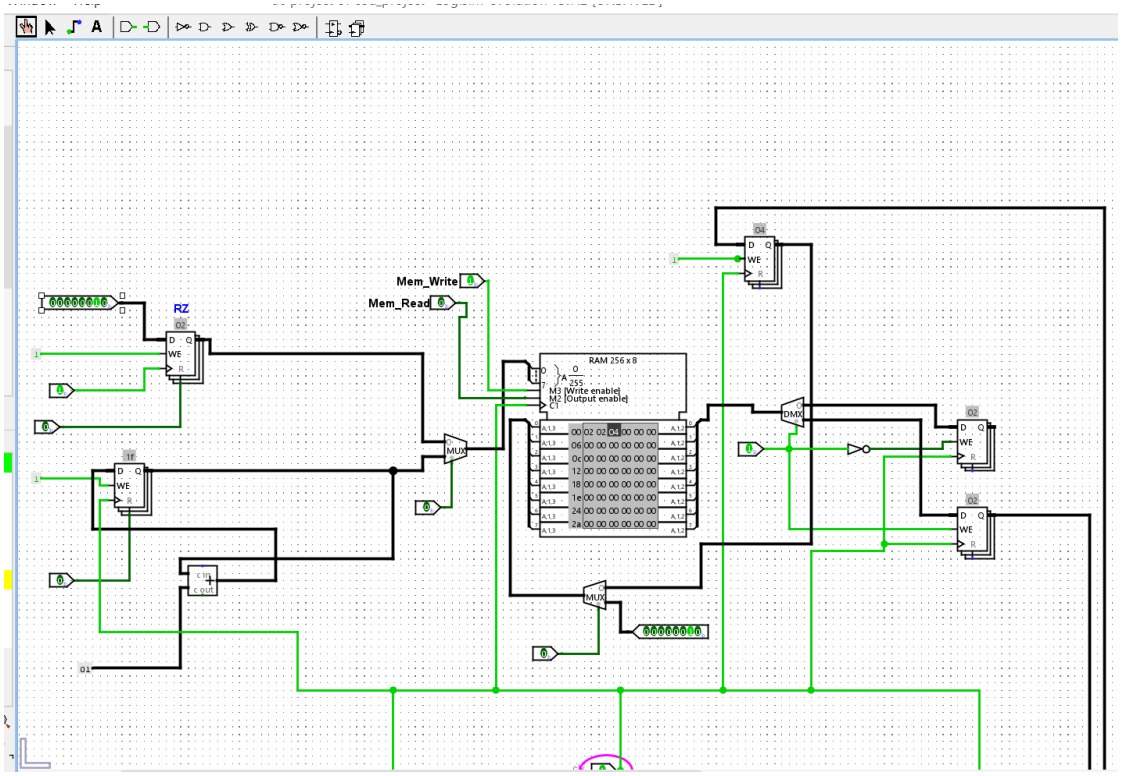
Step 6: Write Back

- Result is written back to the destination register
- PC increments to next instruction

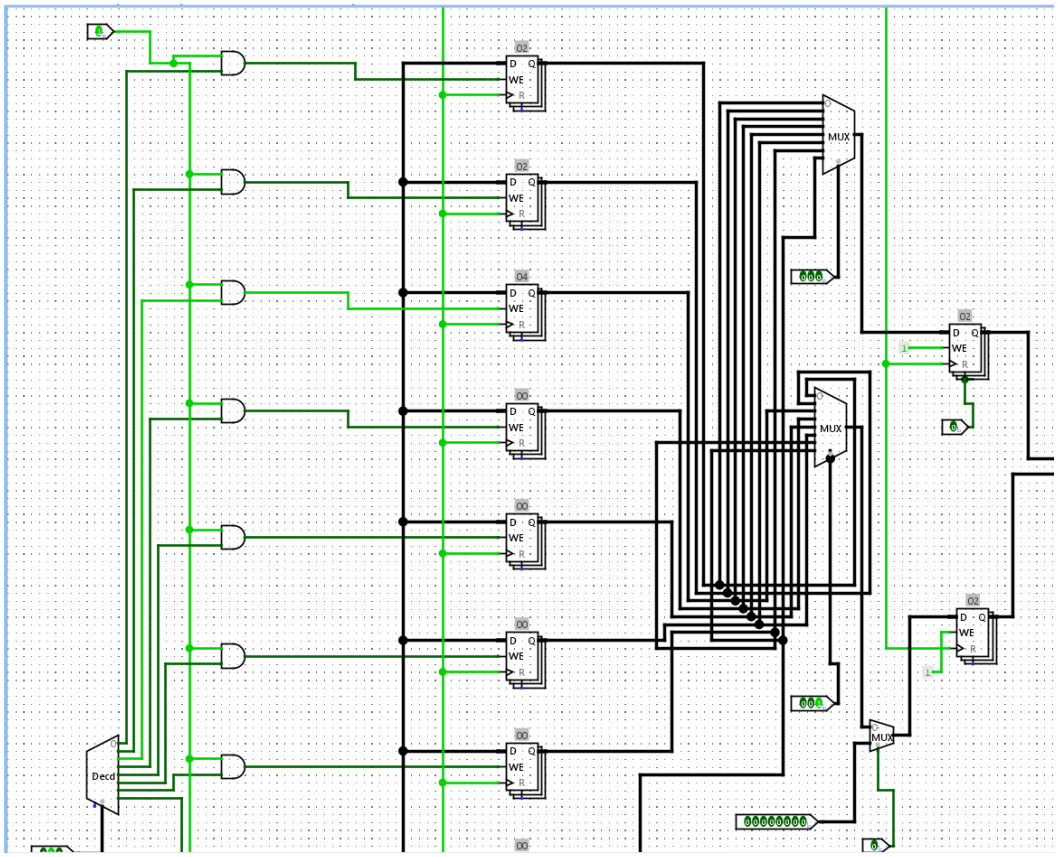
All these steps can be visually verified in Logisim using signal highlights.

---

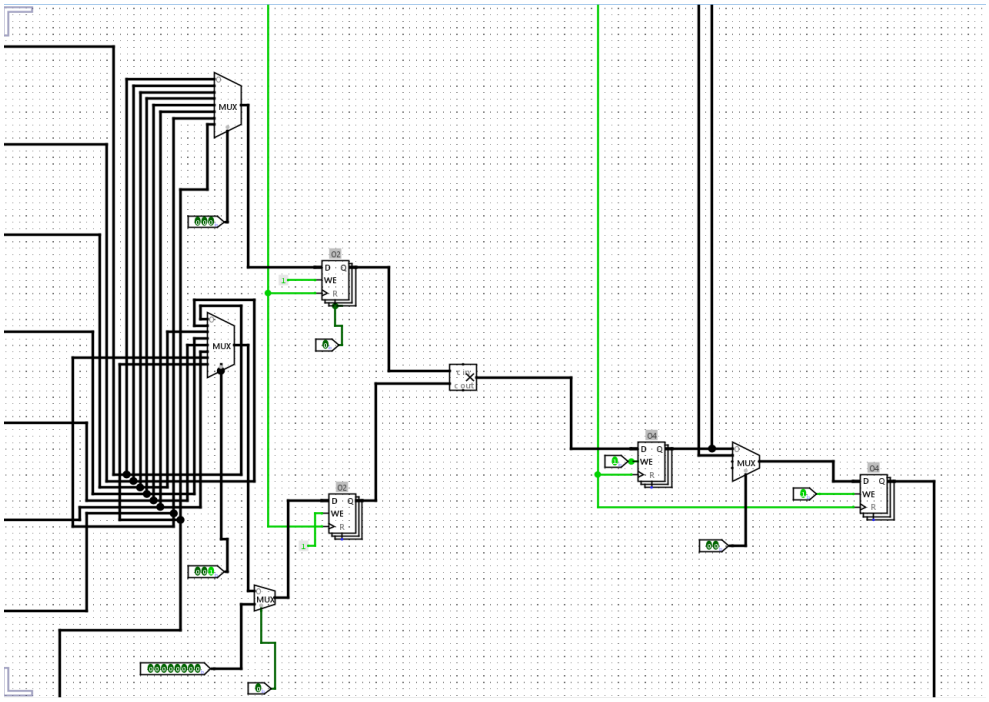
6. Simulation & Results



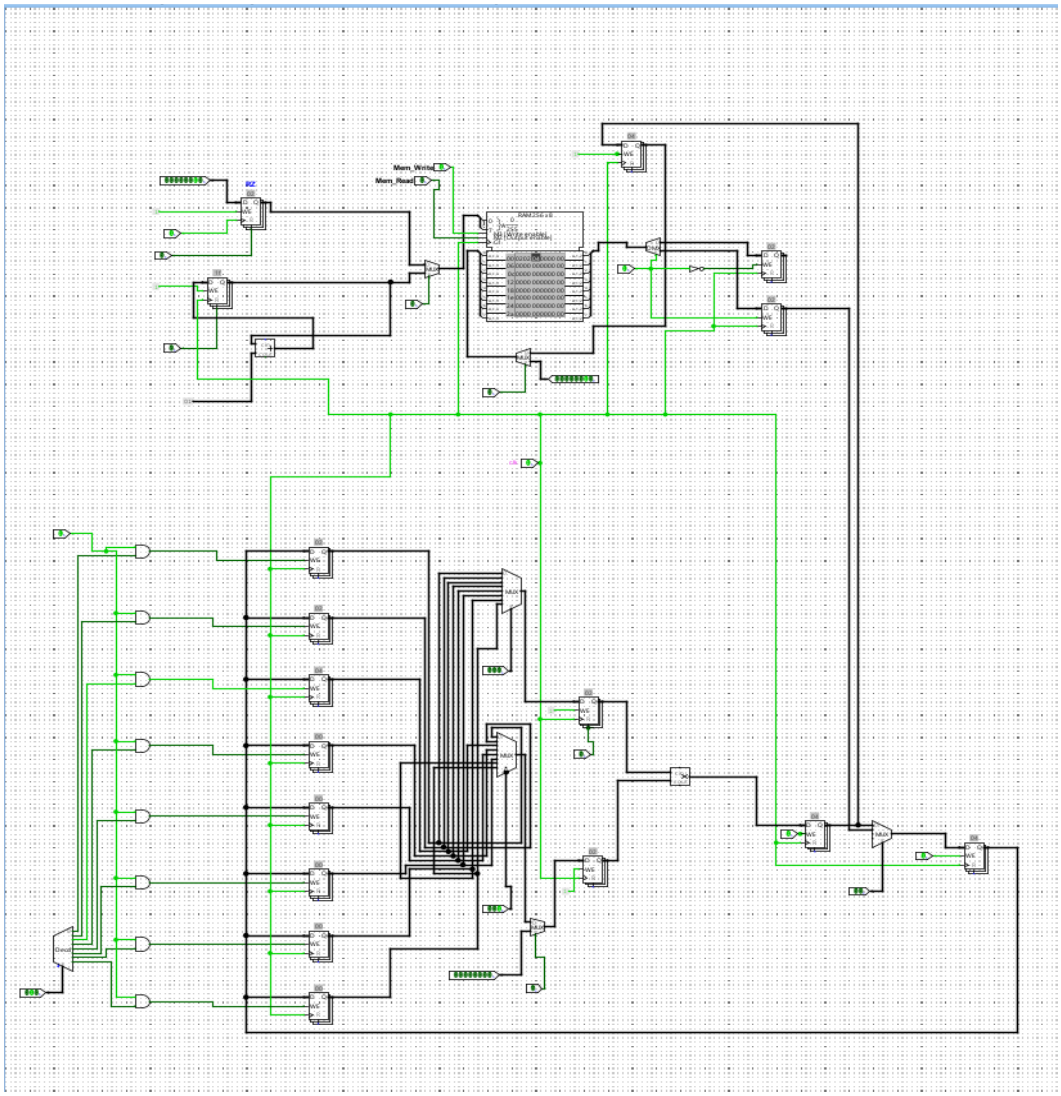
1a. Memory fetching and write backing



1b. Decoding and 8 bit resistor files



1c. ALU part (multiplication of operands):



1d. Overall Processor Design

Observed Results

Operand A = 2
Operand B = 2
ALU Output = 2 * 2 = 4
Therefore,

- Correct execution of arithmetic operations
- Successful memory read/write operations
- Proper register updates after instruction execution
- Accurate data flow as per instruction type

Verification

Each instruction was tested individually, and the output matched expected results, confirming correct processor behavior.

---

7. Conclusion

This project successfully demonstrates the design and implementation of a basic processor using Logisim.
Through this project, we learned:

- Internal working of a processor
- Role of control signals and buses
- Instruction execution cycle
- Practical implementation of COA concepts

The processor can be further enhanced by adding:

- Pipelining
- Interrupt handling
- Branch instructions
- Expanded instruction set

This project serves as a strong foundation for understanding microprocessor and CPU design.